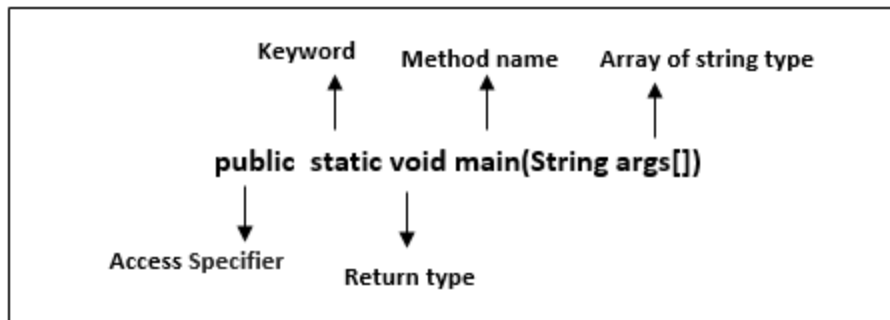


Java

Main Method in Java :-

- In Java, the main() method is the entry point of any Java program.
- Without it, the program won't run (unless using special frameworks)



- **public** -
 - Access modifier.
 - Allows the method to be called by JVM from anywhere.
- **static** -
 - The method belongs to the class, not to an object.
 - JVM can call the main method without creating an object.
- **void** -
 - Return type of the method.
 - Main method does not return any value.
- **main** -
 - Name of the method.
 - This exact name is required for program execution.
- **String[] args** -
 - Parameter to the main method.
 - Used to accept command-line arguments.
 - args is an array of String.
- **{ }** -
 - Curly braces define the body of the method.
 - All code to be executed goes inside { }.

Operators in Java :-

- Operators are special symbols in Java.
- They are used to perform operations on variables and values.
- Java provides different types of operators for various tasks.

1. Arithmetic Operator –

- Arithmetic operators are used for mathematical calculations.

Arithmetic	
Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
++	Increment
--	Decrement

Example –

```
package Basic_Concept;
```

```
public class arithmetic_operation {  
    public static void main(String [] args)  
    {  
        System.out.println("Addition is: "+(5+3));  
        System.out.println("Substraction is: "+(5-3));  
        System.out.println("Multiplication is: "+(5*3));  
        System.out.println("Division is: "+(6/2));  
        System.out.println("Modulus is: "+(6%2));  
    }  
}
```

Output –

Addition is:8

Substraction is:2

Multiplication is:15

Division is:3

Modulus is:0

2. Comparison Operator –

- Comparison Operator used to compare two values and return a boolean result (true/false).

Comparison	
Operator	Description
==	Is equal to
===	Identical
!=	Not equal to
!==	Not Identical
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to

Example –

```
package Basic_Concept;
```

```
public class operator {  
    public static void main(String [] args)  
    {  
        System.out.println("Equals:"+(6==9));  
        System.out.println("Not Equals:"+(6!=9));  
        System.out.println("Greater than:"+(6>9));  
        System.out.println("Less than:"+(6<9));  
        System.out.println("Greater Equals:"+(6>=9));  
        System.out.println("Less Equals:"+(6<=9));  
    }  
}
```

Output –

Equals:false

Not Equals:true

Greater than:false

Less than:true

Greater Equals:false

Less Equals:true

3. Logical Operator –

- Logical operators are used in conditional expressions.

Logical	
Operator	Description
&&	Logical AND
	Logical OR
!	Logical Not

Example –

```
package Basic_Concept;
```

```
public class operator {  
    public static void main(String [] args)  
    {  
        int age=30;  
        System.out.println("Logical AND:"+"(9<3) && (10==2)");  
        System.out.println("Logical OR:"+"(9<3) || (10==2)");  
        System.out.println(!(age > 30));  
    }  
}
```

Output –

Logical AND:false

Logical OR:false

true

4. Assignment Operator –

- Assignment operators assign values to variables.

Operator	Operation	Equivalent to
=	num = 5	num = 5
+=	num+=5	num = num+5
-=	num-=5	num = num-5
=	num=5	num = num*5
/=	num/=5	num = num/5
%=	num%=5	num = num%5

Example –

```
package Basic_Concept;

public class assignment_operator {
    public static void main(String[] args) {
        int a = 10;
        int b = 5;
        System.out.println("a = " + a);
        a += b;
        System.out.println("a += b : " + a);
        a -= b;
        System.out.println("a -= b : " + a);
        a *= b;
        System.out.println("a *= b : " + a);
        a /= b;
        System.out.println("a /= b : " + a);
        a %= b;
        System.out.println("a %= b : " + a);    // 0
    }
}
```

Output –

```
a = 10
a += b : 15
a -= b : 10
a *= b : 50
a /= b : 10
a %= b : 0
```

5. Bitwise Operator –

- Bitwise operators operate on bits of numbers.

Operators	Description	Use
&	Bitwise AND	op1 & op2
	Bitwise OR	op1 op2
^	Bitwise Exclusive OR	op1 ^ op2
~	Bitwise Complement	~op
<<	Bitwise Shift Left	op1 << op2
>>	Bitwise Shift Right	op1 >> op2
>>>	Bitwise Shift Right zero fill	op1 >>> op2

Example –

```
package Basic_Concept;
```

```
public class bitwise_operator {  
  
    public static void main(String[] args) {  
  
        int a = 5;  
  
        int b = 3;  
  
  
        System.out.println("a & b (AND) : " + (a & b));  
  
        System.out.println("a | b (OR) : " + (a | b));  
  
        System.out.println("a ^ b (XOR) : " + (a ^ b));  
  
        System.out.println("~a (NOT) : " + (~a));  
  
        System.out.println("a << 1 (Left Shift): " + (a << 1));  
  
        System.out.println("a >> 1 (Right Shift): " + (a >> 1));  
  
    }  
}
```


Output –

$a \& b$ (AND) : 1

$a | b$ (OR) : 7

$a \wedge b$ (XOR) : 6

$\sim a$ (NOT) : -6

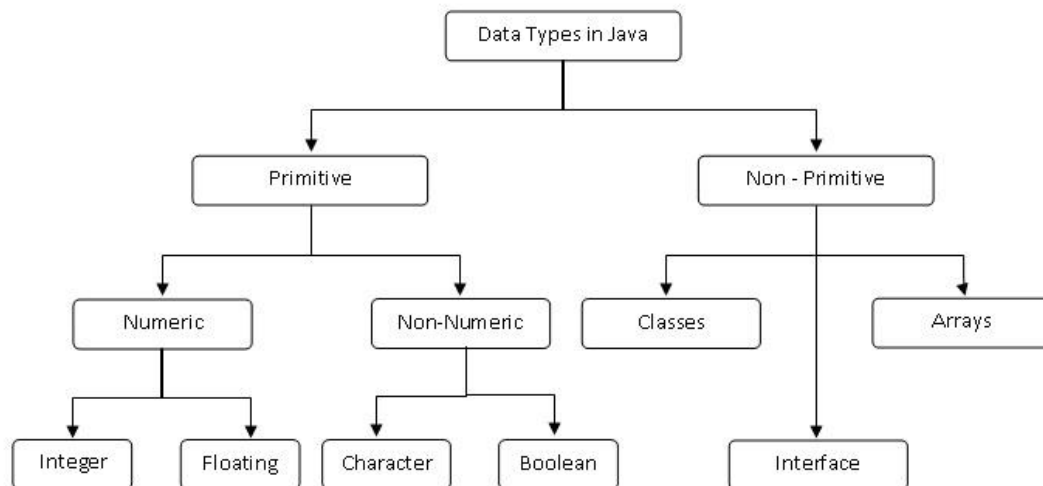
$a \ll 1$ (Left Shift): 10

$a \gg 1$ (Right Shift): 2

Data Type in Java :-

- In Java, data types specify the type of data that a variable can hold.
- They are mainly divided into two categories:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types



1. Primitive Data Types :-

1) **Byte** – Used for saving memory in large arrays.

- Size: 1 byte (8-bit)
- Range: -128 to 127

Example –

byte num = 100;

2) **Short** - Used when memory saving is required.

- Size: 2 bytes (16-bit)
- Range: -32,768 to 32,767

Example –

short value = 32000;

3) **int** - Default data type for integers.

- Size: 4 bytes (32-bit)
- Range: -2,147,483,648 to 2,147,483,647

Example –

```
int salary = 50000;
```

4) long - Used for very large numbers.

- Size: 8 bytes (64-bit)
- Range: -2^{63} to $2^{63} - 1$

Example –

```
long population = 1000000000L;
```

5) float - Must add f or F at the end.

- Size: 4 bytes (32-bit)
- Stores decimal numbers (up to 7 digits precision).

Example –

```
float pi = 3.14f;
```

6) double - Default data type for floating-point values.

- Size: 8 bytes (64-bit)
- Stores decimal numbers (up to 15–16 digits precision).

Example –

```
double price = 99.99;
```

7) Char - Can store letters, digits, or symbols.

- Size: 2 bytes (16-bit, Unicode)
- Stores a single character.

Example –

```
char grade = 'A';
```

8) boolean - Used for conditional checks.

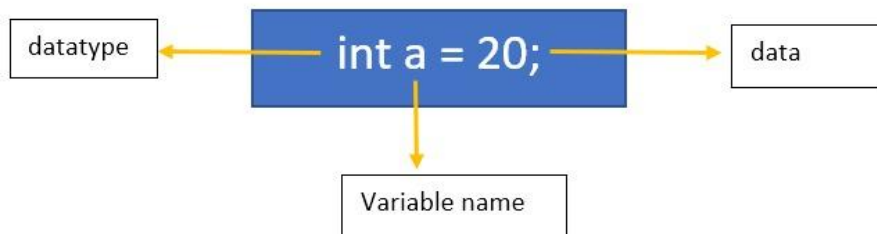
- Size: 1 bit (JVM dependent)
- Values: true / false

Example –

```
boolean isJavaFun = true;
```

Variable in Java :-

- A variable is a name given to a memory location.
- It is used to store data that can be changed during program execution.
- Every variable has a data type, which defines the kind of data it can hold (int, float, char, etc.).



Rules of Variable Declaration –

- It starts with only characters & underscore.

Example –

```
int age = 20;
```

```
int _value = 100;
```

- It can't start with number, special characters & keyword.

Example –

```
int 1num = 10;
```

```
int @data = 20;
```

```
int class = 30;
```

- It can be alphanumeric.

Example –

```
int rollNo1 = 25;
```

```
int student2025 = 50;
```

- It doesn't contain any spaces.

Example –

```
int student name = "Raj";
```

```
int studentName = "Raj";
```

➤ It should be meaningful.

Example –

```
int marks = 90;
```

Syntax of Variable Declaration –

```
dataType variableName;
```

Example –

```
byte age=23;
```